HIRATA

Fleet Management System

Informe Técnico — Etapa 1

Sistema de Gestión de Flota

Benjamín Alvarez Patricio López Víctor Llanos
Duvan Alfonso Cuta Eduardo Fernández

Integración de Competencias I | José Fuentes Verdugo

IP Santo Tomás — Valdivia | Abril 2026

Contenido

1. Contexto y Objetivos
2. Arquitectura del Sistema
3. Base de Datos
4. Módulos Funcionales
5. Pruebas Automatizadas
6. Conclusión

Contexto del Proyecto

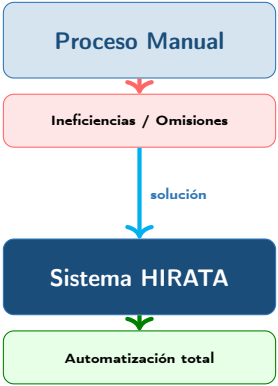
Problema Identificado

Transportes Hirata operaba su flota de camiones con procesos **manuales**:

- Asignación de conductores sin validación
- Sin trazabilidad de kilometraje
- Mantenimiento preventivo omitido con frecuencia
- Sin alertas automáticas de servicio

Solución Propuesta

Sistema web full-stack que **automatiza** el ciclo de vida completo de la flota con API REST segura y notificaciones en tiempo real.



Objetivos y Alcance

Objetivo General

Desarrollar un sistema web que permita administrar asignaciones, kilometraje y mantenimiento preventivo de la flota Hirata.

Objetivos Específicos

- **Autenticación segura JWT + Cookies HTTPOnly.**
- **API REST con arquitectura limpia por capas.**
- **Transacciones ACID para asignaciones de camiones.**
- **Alertas automáticas al superar umbral de 5,000 km.**
- **Ciclo de vida de mantenimiento (Prog/Curso/Fin).**
- **Notificaciones automáticas por rol de usuario.**
- **Pruebas unitarias automatizadas con Jest.**

Alcance Etapa 1

- RF-03: Notificación de mantenimiento.
- RF-04: Gestión de órdenes de servicio.
- RF-05: Especificación de pruebas unitarias.
- Backend funcional y DB completa.

Fuera de Alcance

- Telemetría GPS en tiempo real.
- Pasarela de pagos o facturación.

Stack Tecnológico

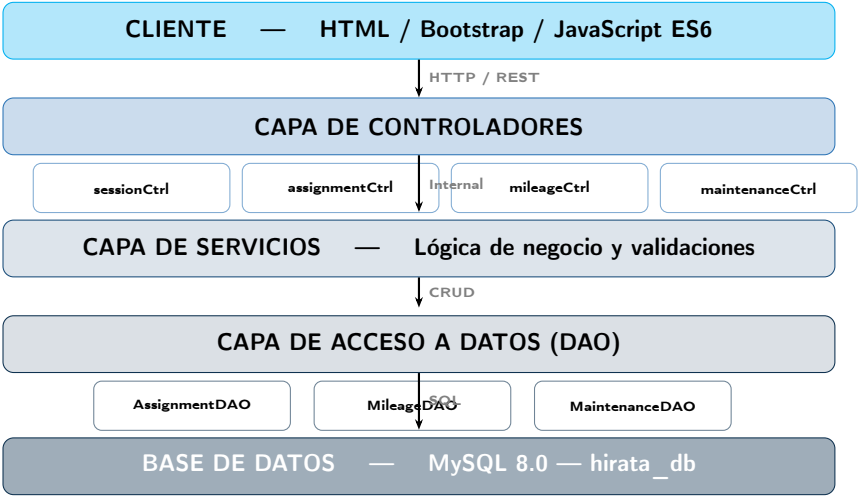
Capa	Tecnología	Rol en el Proyecto
Frontend	HTML5 + Bootstrap 5	Interfaces responsivas (escritorio/móvil)
Lógica UI	JavaScript ES6+	Módulos async/await, gestión de sesiones
Backend	Node.js + Express 5	Servidor API REST, ESModules
Auth	JWT + bcrypt 6	Sesiones seguras, hash de contraseñas
Base de Datos	MySQL 8.0 (mysql2)	Transacciones ACID, pool de conexiones
Testing	Jest + Supertest	Pruebas de integración de rutas
Dev Tools	nodemon + dotenv	Hot-reload, variables de entorno

Node.js + Express

MySQL 8.0

Jest + Supertest

Arquitectura de Tres Capas

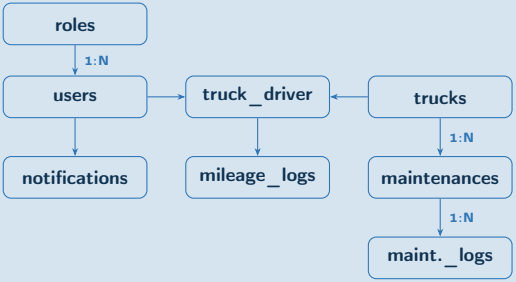


Modelo Relacional — hirata_db

Tablas Principales

Tabla	Descripción
roles	Catálogo de roles del sistema
users	Usuarios con credenciales y rol
trucks	Flota con odómetro y estado
truck_driver	Asignaciones conductor-camión
mileage_logs	Historial de recorridos
maintenances	Órdenes de mantenimiento
maintenance_logs	Bitácora de cada servicio
notifications	Alertas automáticas del sistema

Diagrama E-R Simplificado



Detalle Técnico

Campo status en trucks usa ENUM: *disponible* / *en uso* / *en mantenimiento*. Actualizado automáticamente por la lógica de negocio.

Autenticación y Control de Acceso (RBAC)

Flujo de Autenticación

1. `POST /api/sessions/login` + **Bcrypt verify.**
2. **Firma de JWT (1h) almacenado en Cookie HTTPOnly.**
3. **Protección contra ataques XSS y CSRF.**

Roles del Sistema

Rol	Permisos
superadmin	**Control total de usuarios y sistema.**
admin	**Gestión de flota y reportes.**
driver	**Registro de recorridos.**
maint	**Ejecución de órdenes técnicas.**

Seguridad

- **Passwords hasheados con Bcrypt.**
- **RBAC mediante middleware** `authorize()`.
- **Sanitización de entradas en la API.**

Middleware authorize()
Valida JWT → Chequea Rol
401 (No Auth) / 403 (No Permisos)

RF-03: Registro de Kilometraje y Notificaciones

Flujo Transaccional

1. Validar asignación activa del conductor
2. Verificar estado del camión (*no en mantenimiento*)
3. Validar coherencia: nuevo km > odómetro actual
4. Insertar en `mileage_logs`
5. Calcular: $\Delta km = km_{nuevo} - km_{ltimo_servicio}$
6. Si $\Delta km \geq 5{,}000$: **activar alerta**
7. Actualizar `total_mileage` y `status` del camión
8. **INSERT masivo** de notificaciones a admin/maintenance

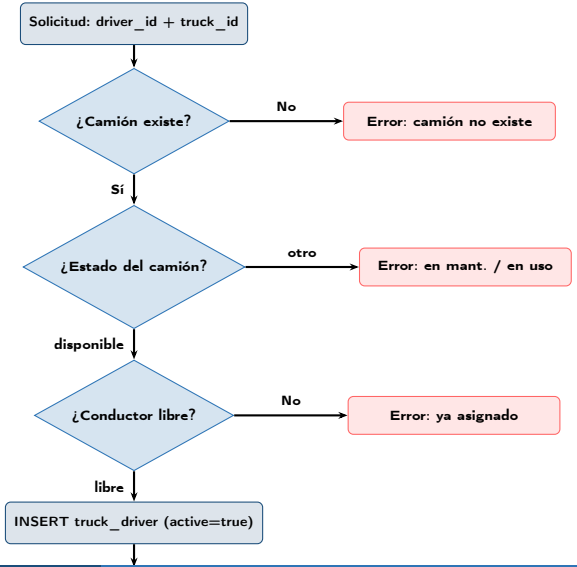
Lógica de Umbral

```
const delta = mileage_value - truck.last_maintenance_mileage; const
needsMaint = delta >= truck.maintenance_threshold;
// Actualizar camion UPDATE trucks SET total_mileage=?, status=? WHERE
id=? // 'en mantenimiento' o 'disponible'
// INSERT masivo notificaciones SELECT id FROM users WHERE role_id IN
(1,2,4) AND active = true
```

Umbral por defecto: **5.000 km**.

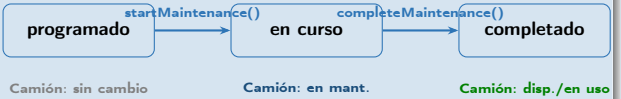
Configurable por camión (`maintenance_threshold`).

RF-03: Flujo de Asignación de Camiones



RF-04: Gestión de Mantenimiento

Ciclo de Vida de una Orden



Finalización Inteligente

Al completar, el sistema consulta si existe una asignación activa: si la hay, el camión vuelve a `'en uso'`; si no, pasa a `'disponible'`.

API del Módulo

Endpoint	Acción
`POST /maintenances`	**Crear orden**
`GET /maintenances`	**Listar todas**
`GET /maintenances/:tid`	**Por camión**
`PUT /maintenances/:id/start`	**Iniciar**
`PUT /maintenances/:id/complete`	**Completar**
`PUT /maintenances/:id`	**Actualizar**
`DELETE /maintenances/:id`	**Eliminar**

Sistema de Notificaciones

- Modo **pasivo**: disparado por mileage/maint
- Tipos: mantenimiento, asignacion, sistema, error
- Destinatarios por rol (admin, superadmin, maint)

RF-05: Pruebas Unitarias con Jest

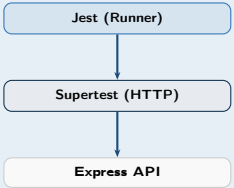
Suites de Prueba

Archivo	Cobertura de Código
`auth.test.js`	**Middleware RBAC y protección de rutas.**
`sessions.test.js`	**Login, manejo de cookies y logout.**
`mileage.test.js`	**Lógica de umbrales y alertas de km.**
`trucks.test.js`	**CRUD y validación de estados físicos.**
`users.test.js`	**Gestión de perfiles y permisos 403.**

Ejecución

1. **Comando:** `npm test` **en el directorio** `server/`.
2. **Mocking de Base de Datos para entornos aislados.**
3. **Validación de códigos de estado HTTP (2xx, 4xx, 5xx).**

Arquitectura de Testing



Regla Crítica

Cualquier cambio en la lógica de negocio debe superar las pruebas de integración antes del *merge* para asegurar la estabilidad del sistema.

Logros de Etapa 1

Hitos Completados

- ✓ **DB Relacional normalizada en MySQL.**
- ✓ **Arquitectura Backend por capas (DAO/Service).**
- ✓ **Lógica de umbrales para mantenimiento.**
- ✓ **Sistema de notificaciones transversales.**
- ✓ **Control de estados automático en camiones.**

Trabajo Pendiente

- **Implementar suites Jest (RF-05).**
- **Refinar interfaces del Frontend.**

Aspectos Técnicos Destacados

Consistencia: Uso de `START TRANSACTION` en asignaciones para evitar datos huérfanos.

Escalabilidad: DAO independiente de la lógica de negocio.

Seguridad: Cookies *secure* para evitar robo de tokens.

¡Gracias!

HIRATA Fleet Management System

Integración de Competencias I — Etapa 1

Benjamín Alvarez Patricio López Víctor Llanos
Duvan Alfonso Cuta Eduardo Fernández

Docente: José Fuentes Verdugo
IP Santo Tomás — Valdivia, Chile | Abril 2026

github.com/duvalfonso/integracion-de-competencias